

D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY, KOLHAPUR

(An Autonomous Institute)



DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

**A
PROJECT REPORT ON**

“SMART CAR PARKING SYSTEM”

SUBMITTED BY:

NAME	ROLL NO.
Pratiksha P Vhanmane	29
Madhura D Patil	38
Rani S Dargude	40

Research Paper Guide

Miss.P.S.Pise

HOD

Dr. T.B. Mohite-Patil

INTRODUCTION

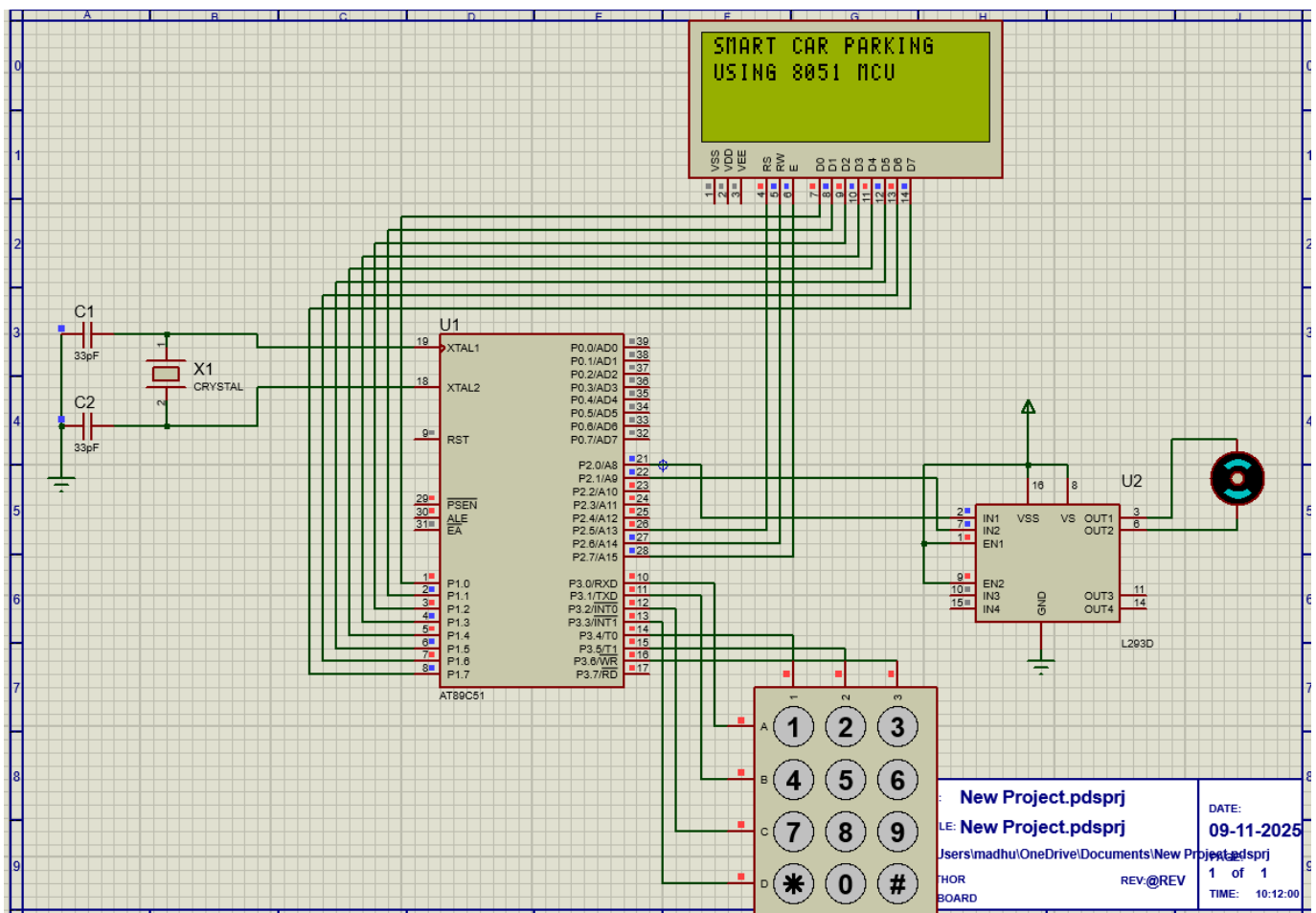
With the increasing number of vehicles, efficient parking management has become an important requirement in residential and commercial areas. Traditional parking systems often face problems such as unauthorized access, inefficient slot utilization, and difficulty in monitoring vehicle entry and exit.

The Smart Car Parking System using AT89C51 microcontroller is designed to provide a secure and automated parking solution. The system uses a keypad for password-based authentication, a 20×4 LCD for displaying user information and parking slot status, and an L293D motor driver to control the parking gate mechanism.

Each user is assigned a unique access code. When a valid code is entered, the corresponding parking slot is allocated, and the gate is operated automatically. Re-entering the same code releases the occupied slot. Invalid users are denied access, thereby improving security and efficient management of parking spaces.

This project demonstrates how embedded systems can be effectively used for access control, automation, and smart parking management. The simulation is carried out using Proteus Design Suite, and the embedded program is developed using Keil μ Vision IDE.

CIRCUIT DIAGRAM



CIRCUIT DESCRIPTION:

1. Microcontroller (AT89C51):

Acts as the brain of the system. Controls the LCD, keypad, and motor driver (L293D). It reads the input code entered via the keypad, compares it with the stored codes, and performs gate control operations. Operates at 5V DC with an 11.0592 MHz crystal oscillator connected across XTAL1 and XTAL2 pins for accurate timing.

2. Power Supply Section:

The entire circuit works on a regulated +5 V DC supply. A 7805 voltage regulator (if implemented) ensures a stable voltage for the 8051, LCD, and other peripherals.

3. Keypad (4×3 Matrix):

Used to simulate RFID card input by entering a 4-digit user code. Connected to Port 3 (P3.0 – P3.6) of the microcontroller. When a key is pressed, the corresponding row and column lines are shorted, which the microcontroller scans to identify the pressed key.

4. LCD Display (20×4):

Connected to Port 1 for data lines (D0–D7) and P2.5–P2.7 for control pins (RS, RW, EN). Displays system messages such as “RFID Parking System,” “Enter User Code,” “Welcome Sharma,” and real-time slot status (Occupied/Empty).

5. Motor Driver (L293D):

Used to control the DC motor that represents the parking gate barrier. Takes logic signals from the microcontroller’s P2.0 and P2.1 pins. Allows the motor to rotate in forward (open gate) or reverse (close gate) directions. Provides current amplification since the 8051’s I/O pins cannot drive a motor directly.

6. DC Motor:

Simulates the movement of the parking gate. When the code is valid, the motor rotates in one direction (gate opens); after a short delay, it rotates in the opposite direction (gate closes).

7. Crystal Oscillator and Capacitors:

A 11.0592 MHz crystal connected across XTAL pins with two 33 pF capacitors ensures stable clock pulses for microcontroller timing and serial operations.

8. Reset Circuit:

Consists of a 10 μ F capacitor and 10 k Ω resistor connected to the RST pin. Ensures the microcontroller starts from a known state when powered on.

9. Slot Indication Logic (in LCD):

The last line of the LCD continuously shows slot status as ☒ for occupied and ☐ for empty. Each valid user code toggles a slot between occupied and free, reflecting real-time parking availability.

10. Simulation Tools:

The entire circuit is designed and simulated in Proteus Design Suite. Embedded C code is written and compiled using Keil μ Vision IDE and the resulting HEX file is loaded into the AT89C51 in Proteus.

KEIL C CODE:

```
#INCLUDE <REG51.H>
/* ===== PIN DEFINITIONS ===== */
#define LCD P1
SBIT RS = P2^5;
SBIT RW = P2^6;
SBIT EN = P2^7;
SBIT MOTOR_P = P2^0;
SBIT MOTOR_N = P2^1;
/* KEYPAD ROWS AND COLUMNS */
SBIT R1 = P3^0;
SBIT R2 = P3^1;
SBIT R3 = P3^2;
SBIT R4 = P3^3;
SBIT C1 = P3^4;
SBIT C2 = P3^5;
SBIT C3 = P3^6;
/* VARIABLES */
UNSIGNED CHAR CODE_IN[5];
BIT SLOT1 = 0, SLOT2 = 0, SLOT3 = 0, SLOT4 = 0;
/* == FUNCTION PROTOTYPES == */
VOID DELAY_MS(UNSIGNED INT);
VOID LCD_CMD(UNSIGNED CHAR);
VOID LCD_DATA(UNSIGNED CHAR);
VOID LCD_PRINT(CHAR *);
VOID LCD_INIT(VOID);
CHAR GET_KEY(VOID);
BIT CHECK_CODE(CHAR *INPUT, CHAR *VALID);
VOID SHOW_SLOTS(VOID);
/* ===== MAIN PROGRAM ===== */
VOID MAIN(VOID){
    UNSIGNED CHAR I;
    CHAR K;
    MOTOR_P = MOTOR_N = 0;
    LCD_INIT();
    /* STARTUP MESSAGE */
    LCD_CMD(0x01);
    LCD_PRINT("SMART CAR PARKING ");
    LCD_CMD(0xC0);
    LCD_PRINT("USING 8051 MCU");
    DELAY_MS(1500);
    WHILE (1) {
        LCD_CMD(0x01);
        LCD_PRINT("ENTER USER CODE:");
        LCD_CMD(0xC0);
        LCD_PRINT("CODE: ");
```

```

LCD_CMD(0x94);
LCD_PRINT("SLOTS:");
SHOW_SLOTS();
LCD_CMD(0xC6); // AFTER "CODE: "
FOR (I = 0; I < 4; ) {
    K = GET_KEY();
    IF (K != 0) {
        CODE_IN[I++] = K;
        LCD_DATA('*');
    }
}
CODE_IN[4] = '\0';
DELAY_MS(300);
LCD_CMD(0x01);
/* ===== CODE CHECKS ===== */
IF (CHECK_CODE(CODE_IN, "1234"))
{
    IF (!SLOT1) {
        SLOT1 = 1;
        LCD_PRINT("WELCOME SHARMA");
        LCD_CMD(0xC0);
        LCD_PRINT("SLOT 1 OPENED");
        MOTOR_P = 1; MOTOR_N = 0; DELAY_MS(800);
    }
    ELSE {
        LCD_PRINT("GOODBYE SHARMA");
        LCD_CMD(0xC0);
        LCD_PRINT("SLOT 1 FREED");
        MOTOR_P = 0; MOTOR_N = 1; DELAY_MS(800);
    }
}

    MOTOR_P = 1; MOTOR_N = 0; DELAY_MS(800);
}
ELSE {
    SLOT2 = 0;
    LCD_PRINT("GOODBYE MURTHY");
    LCD_CMD(0xC0);
    LCD_PRINT("SLOT 2 FREED");
    MOTOR_P = 0; MOTOR_N = 1; DELAY_MS(800);
}
}
}
DELAY_MS(800);
ELSE {
    SLOT3 = 0;
    LCD_PRINT("GOODBYE KHAN");
    LCD_CMD(0xC0);

```

```

        LCD_PRINT("SLOT 3 FREED");
        MOTOR_P = 0; MOTOR_N = 1; DELAY_MS(800);
    }
}
ELSE IF (CHECK_CODE(CODE_IN, "4567")){
    IF (!SLOT4){
        SLOT4 = 1;
        LCD_PRINT("WELCOME REDDY");
        LCD_CMD(0xC0);
        LCD_PRINT("SLOT 4 OPENED");
        MOTOR_P = 1; MOTOR_N = 0; DELAY_MS(800);
    }
    ELSE {
        SLOT4 = 0;
        LCD_PRINT("GOODBYE REDDY");
        LCD_CMD(0xC0);
        LCD_PRINT("SLOT 4 FREED");
        MOTOR_P = 0; MOTOR_N = 1; DELAY_MS(800);
    }
}
ELSE {
    LCD_PRINT("INVALID CODE");
    LCD_CMD(0xC0);
    LCD_PRINT("ACCESS DENIED");
}
MOTOR_P = 0; MOTOR_N = 0;
DELAY_MS(1200);
LCD_PRINT("SLOTS:");
SHOW_SLOTS();
DELAY_MS(1000);
}
}
/* ===== FUNCTIONS ===== */
VOID SHOW_SLOTS(VOID){
    LCD_CMD(0xD4); // 4TH LINE
    IF (SLOT1) LCD_PRINT("[! ]"); ELSE LCD_PRINT("[ ]");
    IF (SLOT2) LCD_PRINT("[! ]"); ELSE LCD_PRINT("[ ]");
    IF (SLOT3) LCD_PRINT("[! ]"); ELSE LCD_PRINT("[ ]");
    IF (SLOT4) LCD_PRINT("[! ]"); ELSE LCD_PRINT("[ ]");
}
/* SIMPLE 4x3 KEYPAD SCANNING */
CHAR GET_KEY(VOID){
    CHAR KEY = 0;
    R1 = R2 = R3 = R4 = 1;
/* DRIVE EACH ROW LOW AND CHECK COLUMNS */
    R1 = 0;
    IF (!C1) { KEY='1'; WHILE(!C1); }

```

```

    IF (!C2) { KEY='2'; WHILE(!C2); }
    IF (!C3) { KEY='3'; WHILE(!C3); }
    R2 = 0;
    IF (!C1) { KEY='4'; WHILE(!C1); }
    IF (!C2) { KEY='5'; WHILE(!C2); }
    IF (!C3) { KEY='6'; WHILE(!C3); }
    R3 = 0;
    IF (!C1) { KEY='7'; WHILE(!C1); }
    IF (!C2) { KEY='8'; WHILE(!C2); }
    IF (!C3) { KEY='9'; WHILE(!C3); }
    R3 = 1;
    IF (!C2) { KEY='0'; WHILE(!C2); }
    R4 = 1;
    RETURN KEY;
}
BIT CHECK_CODE(CHAR *INPUT, CHAR *VALID){
    UNSIGNED CHAR I;
    FOR (I = 0; I < 4; I++)
        IF (INPUT[I] != VALID[I]) RETURN 0;
    RETURN 1;
}
/* LCD FUNCTIONS */
VOID LCD_CMD(UNSIGNED CHAR VAL){
    LCD = VAL;
    RS = 0; RW = 0; EN = 1;
    DELAY_MS(2);
    EN = 0;
}
VOID LCD_DATA(UNSIGNED CHAR VAL){
    LCD = VAL;
    RS = 1; RW = 0; EN = 1;
    DELAY_MS(2);
    EN = 0;
}
VOID LCD_PRINT(CHAR *S){
    WHILE (*S) LCD_DATA(*S++);
}
VOID LCD_INIT(VOID){
    DELAY_MS(20);
    LCD_CMD(0x38);
    LCD_CMD(0x0C);
    DELAY_MS(5);
}
/*DELAY(APPROX 1MS PER LOOP @11.0592MHz) */
VOID DELAY_MS(UNSIGNED INT MS)
    FOR (J=0;J<250;J++);
}

```

OUTPUT & RESULT:

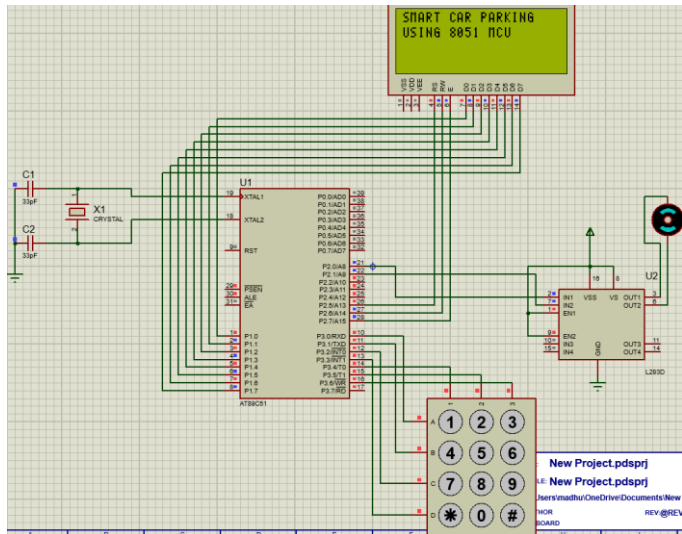


Figure 1: Output Display of Project Title

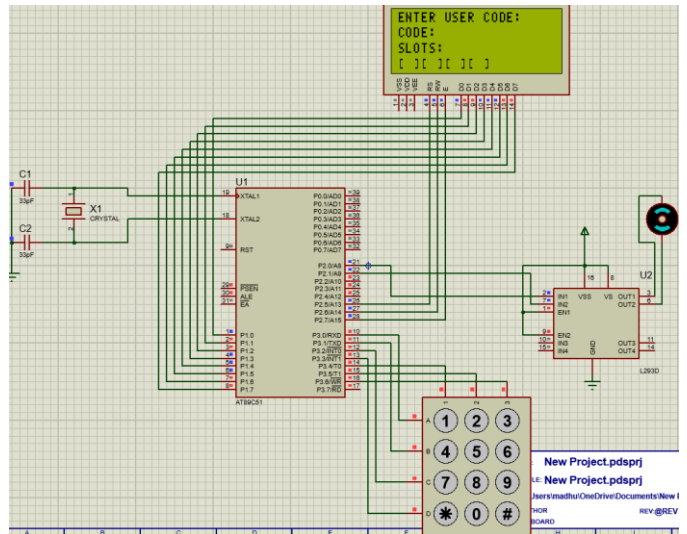


Figure 2: Output Display enter user code screen

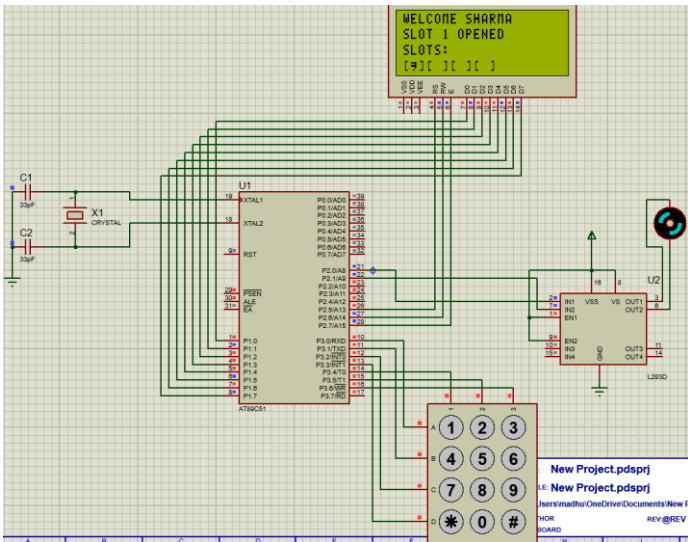


Figure 3: Output Display of Slot 1 has opened

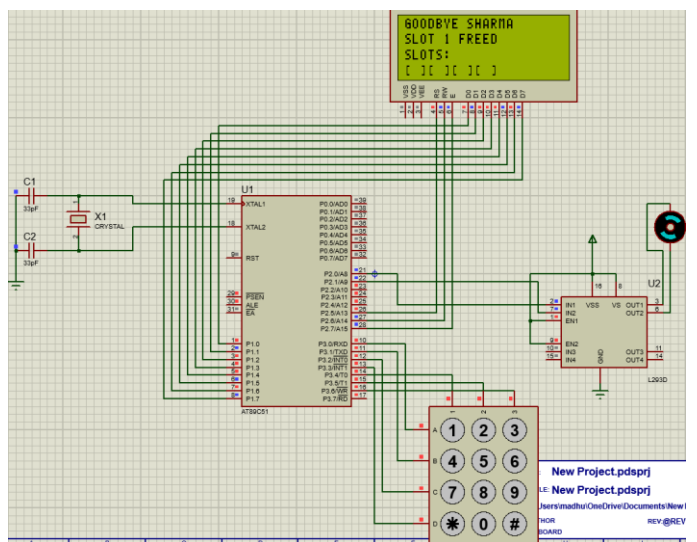


Figure 4: Output Display of slot 1 is free

The Smart Car Parking System was successfully designed and simulated using Proteus and programmed using Embedded C in Keil uVision. The system correctly accepts valid user codes entered through the keypad and displays the corresponding user information on the LCD.

When a valid code is entered, the respective parking slot is allocated and the gate motor is activated through the L293D motor driver. If the same user enters the code again, the occupied slot is released. The LCD continuously displays the status of all parking slots, helping users monitor slot availability.

The system also rejects invalid codes and displays an "ACCESS DENIED" message, thereby improving security. The simulation results confirmed proper operation of keypad interfacing, LCD display, motor control, and parking slot management.

Hence, the proposed Smart Car Parking System provides an efficient, secure, and low-cost solution for automated parking management.

WORKING:

1. SYSTEM INITIALIZATION:

WHEN POWER IS SUPPLIED, THE MICROCONTROLLER INITIALIZES ALL CONNECTED DEVICES — LCD, KEYPAD, AND MOTOR DRIVER. THE LCD FIRST DISPLAYS THE SYSTEM TITLE:

RFID BASED CAR PARKING SYSTEM
USING KEYPAD

AFTER A SHORT DELAY, IT SWITCHES TO THE OPERATIONAL SCREEN SHOWING:

“ENTER USER CODE” ON THE FIRST LINE.

“CODE:” AND THE ENTERED DIGITS ON THE SECOND LINE.

“SLOTS” WITH REAL-TIME AVAILABILITY ON THE THIRD AND FOURTH LINES.

2. USER CODE ENTRY (SIMULATED RFID INPUT):

THE KEYPAD ACTS AS A SIMULATED RFID INPUT DEVICE. THE USER ENTERS A 4-DIGIT UNIQUE CODE ASSIGNED TO THEM USING THE KEYPAD. EACH ENTERED DIGIT IS DISPLAYED AS * ON THE LCD FOR SECURITY.

3. CODE VERIFICATION:

THE MICROCONTROLLER COMPARES THE ENTERED CODE WITH THE STORED VALID CODES IN ITS PROGRAM MEMORY. IF THE CODE MATCHES ANY OF THE STORED RFID USER IDs, ACCESS IS GRANTED. IF THE CODE DOES NOT MATCH, AN “INVALID CODE / ACCESS DENIED” MESSAGE IS DISPLAYED.

4. GATE CONTROL MECHANISM:

WHEN A VALID CODE IS ENTERED, THE CORRESPONDING PARKING SLOT IS MARKED AS OCCUPIED ON THE LCD DISPLAY.

THE MICROCONTROLLER SENDS CONTROL SIGNALS TO THE L293D MOTOR DRIVER, WHICH DRIVES THE DC MOTOR. THE MOTOR ROTATES IN ONE DIRECTION TO OPEN THE GATE AND THEN REVERSES TO CLOSE THE GATE AFTER A SHORT DELAY, SIMULATING VEHICLE ENTRY.

5. SLOT STATUS UPDATE:

THE LCD CONTINUOUSLY SHOWS SLOT AVAILABILITY IN REAL TIME ON THE LAST LINE AS BOXES:

☒ → OCCUPIED SLOT

☐ → EMPTY SLOT

EACH REGISTERED USER HAS A PREDEFINED SLOT (E.G., SHARMA – SLOT 1, MURTHY – SLOT 2, ETC.).

WHEN A VALID USER EXITS (ENTERS THE SAME CODE AGAIN), THE SLOT STATUS TOGGLES BACK TO EMPTY.

6. INVALID CODE HANDLING:

IF A USER ENTERS AN INCORRECT CODE, THE SYSTEM DOES NOT OPERATE THE GATE MOTOR.

THE LCD DISPLAYS:

INVALID CODE
ACCESS DENIED

THE SYSTEM WAITS FOR A NEW VALID INPUT.

7. CONTINUOUS OPERATION:

THE SYSTEM RUNS CONTINUOUSLY, WAITING FOR NEW CODE ENTRIES. IT PROVIDES REAL-TIME MONITORING OF PARKING OCCUPANCY AND GATE CONTROL AUTOMATION WITHOUT MANUAL

SUPERVISION.

8. SIMULATION AND VERIFICATION:

THE DESIGN IS SIMULATED IN PROTEUS DESIGN SUITE WITH A 4×3 KEYPAD, 20×4 LCD, 8051 MICROCONTROLLER, AND DC MOTOR. THE PROGRAM IS DEVELOPED AND COMPILED IN KEIL μ VISION IDE, AND THE GENERATED HEX FILE IS USED FOR SIMULATION TESTING.

KEY FEATURES:

1. Automated Access Control:

The system automatically grants or denies parking access based on user authentication through keypad entry.

2. Microcontroller-Based Operation:

The entire system is controlled by an 8051 microcontroller, which handles input scanning, code verification, LCD display, and motor control.

3. Real-Time Slot Monitoring:

The LCD continuously displays slot status in real time, showing occupied and empty slots using box symbols.

4. Multiple User Support:

The program can store multiple valid user codes, each assigned to a specific parking slot.

5. Secure Authentication:

User codes entered on the keypad are masked as *, ensuring privacy during input.

6. Automatic Gate Control:

The DC motor (through L293D driver) simulates the opening and closing of the parking gate automatically after successful authentication.

7. Error Handling:

Displays “Invalid Code” messages for unauthorized users, preventing access.

8. Low Power Consumption:

The system uses minimal components and operates efficiently on +5V supply, suitable for embedded or battery-based setups.

APPLICATIONS

1. Automated Car Parking Systems – In commercial complexes, offices, and malls for controlled vehicle entry.
2. Residential Apartment Parking – To manage multiple users with assigned slots securely.
3. Smart City Projects – Demonstrates an IoT-compatible parking control model.
4. Institutional and Corporate Campuses – For employee vehicle access control and record keeping.
5. Toll Gates and Access Points – Can be extended for automatic barrier control based on authorized entry.
6. Prototyping and Educational Projects – Useful for students to understand embedded systems and access control logic.
7. Private Garages and Security Gates – For secure gate automation using password or RFID entry.

FUTURE SCOPE:

1. INTEGRATION WITH REAL RFID MODULES:

THE CURRENT SYSTEM USES KEYPAD-BASED INPUT AS A SIMULATION. IT CAN BE UPGRADED BY INTERFACING AN RFID READER (EM-18 OR MFRC522) TO AUTOMATICALLY DETECT USER TAGS.

2. IOT CONNECTIVITY:

THE SYSTEM CAN BE CONNECTED TO THE INTERNET OF THINGS (IOT) USING MODULES LIKE ESP8266 OR NODEMCU TO MONITOR SLOT AVAILABILITY AND USER ACCESS REMOTELY THROUGH A WEB OR MOBILE DASHBOARD.

3. AUTOMATIC VEHICLE DETECTION:

USE OF IR SENSORS OR ULTRASONIC SENSORS CAN AUTOMATICALLY DETECT VEHICLE PRESENCE IN PARKING SLOTS, ELIMINATING MANUAL UPDATES.

4. DATABASE AND TICKETING SYSTEM:

THE DESIGN CAN BE EXTENDED WITH A DATABASE (USING RASPBERRY PI OR PC) TO MAINTAIN USER DETAILS, SLOT RECORDS, AND PARKING DURATION FOR BILLING PURPOSES.

5. SOLAR-POWERED OPERATION:

FOR ENERGY EFFICIENCY, THE SYSTEM CAN BE POWERED USING SOLAR PANELS AND BATTERY BACKUP, MAKING IT ECO-FRIENDLY AND SUITABLE FOR OUTDOOR INSTALLATIONS.

6. LICENSE PLATE RECOGNITION:

WITH IMAGE PROCESSING (VIA RASPBERRY PI OR CAMERA MODULE), THE SYSTEM COULD AUTOMATICALLY READ LICENSE PLATES FOR ENTRY AUTHENTICATION.

7. SMART DISPLAY INTEGRATION:

LARGE DIGITAL SCREENS OR LED MATRICES CAN BE ADDED TO SHOW REAL-TIME PARKING SLOT AVAILABILITY AT THE ENTRANCE OF THE PARKING AREA.

8. VOICE OR APP-BASED CONTROL:

FUTURE VERSIONS CAN ALLOW GATE CONTROL USING MOBILE APPLICATIONS, BLUETOOTH, OR VOICE ASSISTANTS FOR CONVENIENCE.

9. SCALABLE MULTI-GATE SYSTEM:

THE DESIGN CAN BE EXPANDED TO MANAGE MULTIPLE ENTRY AND EXIT POINTS WITH CENTRALIZED MONITORING

Conclusion

The Smart Car Parking System using 8051 Microcontroller successfully demonstrates an efficient and automated method for managing vehicle entry and slot allocation. In this project, a keypad is used to simulate RFID input for user authentication, while the LCD display provides real-time feedback on parking status and slot availability. The system ensures secure access, automatic gate control, and clear visual indication of occupied and empty slots.

This project highlights the practical application of embedded systems in developing smart, automated parking solutions. With further integration of real RFID modules, sensors, and IoT technology, the system can evolve into a fully functional, intelligent parking management system suitable for modern smart cities and commercial complexes.